



Real-Time Crime Analytics & Intelligent Alert System

Course: CS4109 — Fundamentals of Big Data Analytics
Section: CS-A | **Session:** Spring 2026

Member	Name	Roll Number
1	Muhammad Mobin	21I-0794
2	Muhammad Nouman Hafeez	21I-0416

Instructor: Ms. Kainat Iqbal & Dr. Asif Muhammad
Submission: 11 May 2026, 11:59 PM

“Building something that matters — real-time public-safety intelligence for the modern city.”

Contents

1	Introduction	1
2	Learning Objectives Achieved	1
3	Dataset Description	1
3.1	Schema Design and Cleaning	2
4	System Architecture	2
4.1	High-Level Lambda Architecture	3
4.2	Technology Stack	3
5	Batch Layer — Apache Spark	3
5.1	Architecture and Responsibilities	3
5.2	Batch Job Execution	3
5.3	Analytics Computed	4
5.3.1	Crime Trend Analysis	4
5.3.2	Arrest Rate Analysis	5
5.3.3	Geospatial Hotspot Detection (K-Means)	6
5.3.4	Additional Batch Analytics	6
6	Speed Layer — Kafka & Apache Storm	7
6.1	Kafka Producer	7
6.2	Storm Topology Design	7
6.2.1	Component Descriptions	7
6.3	Storm Topology Submission	8
6.4	Storm UI Evidence	9
7	Serving Layer — PostgreSQL & MongoDB	11
7.1	PostgreSQL — Structured Analytics Store	11
7.2	MongoDB — Document Alert Store	11
8	Infrastructure & Containerisation	12
8.1	Docker Compose Services	12
8.2	Key Configuration Parameters	13
9	Streamlit Dashboard	14
10	Results Summary	16
11	Challenges and Solutions	16
11.1	Spark -packages / Ivy Cache in Container	16
11.2	Ghost Alerts — Threshold vs. Simulation Rate	16
11.3	Kafka Consumer Lock on Topology Resubmission	17
11.4	Large Local CSV Files	17
11.5	Dirty Schema Names and Weak Join Keys	17
11.6	Container Orchestration on Windows/WSL2	17
12	Limitations and Future Improvements	17

13 AI Use Disclosure	18
14 Conclusion	18
Project Structure	18

Introduction

Urban crime analysis is a critical challenge for public-safety agencies. Traditional batch reporting systems are insufficient for time-sensitive decision-making. Modern cities require intelligent, real-time analytics pipelines capable of detecting anomalies, identifying hotspots, and dispatching alerts as incidents unfold.

This project implements a **Real-Time Crime Analytics and Intelligent Alert System** powered by the City of Chicago's publicly available public-safety datasets. Using a **Lambda Architecture**, the system simultaneously supports:

- **Batch processing** — historical trend analysis, machine learning, and cross-dataset joins using Apache Spark.
- **Stream processing** — real-time event ingestion, windowed anomaly detection, and alert generation using Apache Kafka and Apache Storm.
- **Serving layer** — structured storage in PostgreSQL and MongoDB, with a live Streamlit dashboard.

The design mirrors production-grade big-data pipelines used by law-enforcement analytics platforms, smart-city initiatives, and public-safety research centres. Crime data is a canonical big-data use case: high row counts, multiple related tables, messy fields, and a need for both *aggregate insight* (policy, planning) and *timely signals* (operational awareness).

Learning Objectives Achieved

1. Designed and implemented a Lambda Architecture combining batch and speed layers.
2. Ingested and preprocessed multi-schema heterogeneous CSV datasets using Apache Spark with explicit **StructType** schemas.
3. Published and consumed streaming events using Apache Kafka producers and topics.
4. Implemented a multi-bolt Apache Storm topology with windowing, filtering, and alert logic.
5. Applied unsupervised machine learning (K-Means clustering) for geospatial hotspot detection.
6. Performed cross-dataset joins across heterogeneous public-safety schemas.
7. Persisted analytics results to PostgreSQL and MongoDB; exposed them via a Streamlit dashboard.
8. Containerised and orchestrated all system components using Docker Compose.

Dataset Description

All datasets are sourced from the *City of Chicago Open Data Portal*. Five datasets were used:

Table 1: Chicago Public Safety Datasets

Dataset	Description	Min Rows
Crime Data (2001–Present)	All reported criminal incidents: crime type, location, date, arrest status, district	50,000
Police Stations	Locations, contacts, and coordinates of all CPD district stations	Full
Arrests	Individual arrest records with charges and statutory references	10,000
Violence Reduction	Victims of homicides and non-fatal shootings with gunshot indicators	10,000
Sex Offenders	Registered sex offenders with block-level address and victim-minor flag	10,000

Schema Design and Cleaning

- All reads use **explicit StructType schemas** so malformed rows land in `_corrupt_record` rather than silently shifting columns.
- CSV ingest uses **PERMISSIVE** mode for resilience; downstream steps filter critical nulls (e.g. `case_number` not null).
- **District** is normalised with `trim`, regex extraction, and `lpad` to three digits so string variants join consistently.
- **Timestamps** are parsed from Chicago’s `MM/dd/yyyy hh:mm:ss a` strings into real `timestamp` columns for grouping.
- **Types**: `Arrest` cast to boolean; `LATITUDE/LONGITUDE` to `double` for geo clustering.
- The Kafka producer **skips rows missing latitude or longitude** so the live map path stays consistent.

System Architecture

The system follows the **Lambda Architecture** pattern with three distinct layers.

High-Level Lambda Architecture

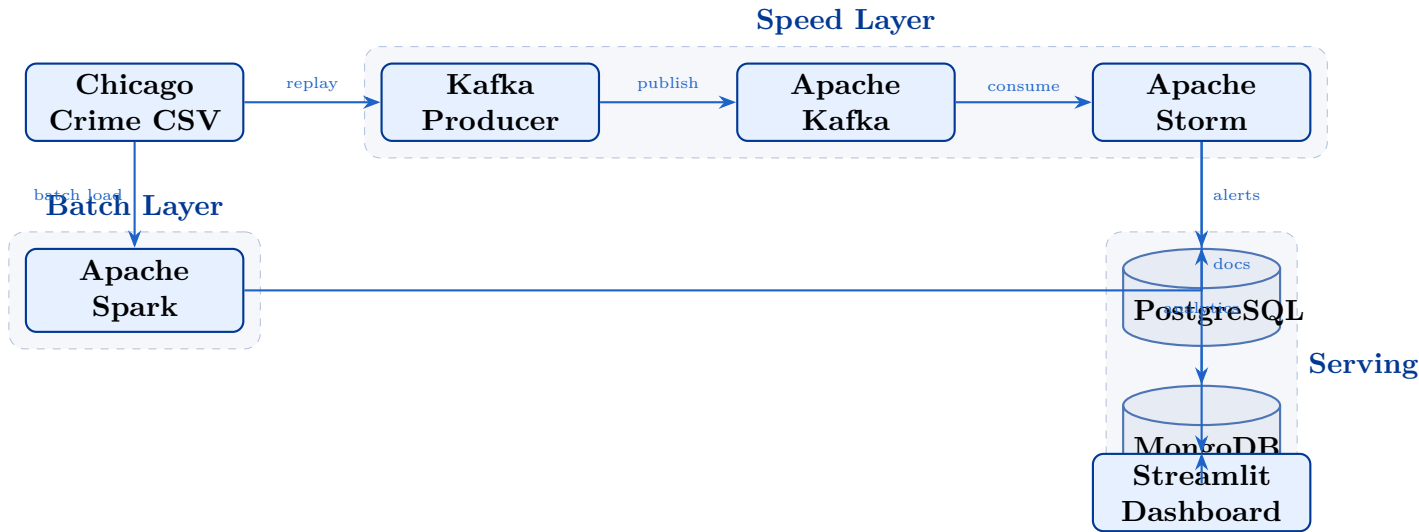


Figure 1: Lambda Architecture — end-to-end data flow

Technology Stack

Table 2: Technology Versions

Component	Technology	Version
Batch Processing	Apache Spark (PySpark)	3.5.0
Stream Broker	Apache Kafka (Confluent)	7.6.1 (cp-kafka)
Stream Processing	Apache Storm	2.6.2
Coordination	Apache ZooKeeper	bundled
Structured Store	PostgreSQL	16
Document Store	MongoDB	7.0
Dashboard	Streamlit	1.41
Producer	Python	3.12
Orchestration	Docker Compose	—

Batch Layer — Apache Spark

Architecture and Responsibilities

The batch layer reads all five datasets from disk, enforces explicit schemas, performs preprocessing, computes analytics, and writes results to PostgreSQL. All DataFrames use explicit `StructType` definitions — schema inference (`inferSchema=True`) was prohibited.

Batch Job Execution

The batch job is submitted via `spark-submit` inside the `spark-master` container:

```
1 docker compose -f docker/docker-compose.yml exec spark-master \
2   /opt/spark/bin/spark-submit \
```

```

3 --master spark://spark-master:7077 \
4 --conf spark.jars.ivy=/tmp/.ivy2 \
5 --packages org.postgresql:postgresql:42.7.4 \
6 /app/spark/batch_job.py \
7 --config /app/config/config.yaml

```

Listing 1: Spark Batch Job Submission Command

```

(.venv) PS D:\CrimeAnalytics> docker compose -f docker/docker-compose.yml exec spark-master /opt/spark/
bin/spark-submit --master spark://spark-master:7077 --conf spark.jars.ivy=/tmp/.ivy2 --packages org.pos
tgresql:postgresql:42.7.4 /app/spark/batch_job.py --config /app/config/config.yaml
4.4 MiB RAM, BlockManagerId(driver, e068f66aac21, 40093, None)
26/05/10 14:31:50 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, e068f66aac21,
40093, None)
26/05/10 14:31:50 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, e068f66aac21, 400
93, None)
26/05/10 14:31:50 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20260510143150-0000/0 on
worker-20260510143103-172.20.0.8-42215 (172.20.0.8:42215) with 2 core(s)
26/05/10 14:31:50 INFO StandaloneSchedulerBackend: Granted executor ID app-20260510143150-0000/0 on hos
tPort 172.20.0.8:42215 with 2 core(s), 1024.0 MiB RAM
26/05/10 14:31:50 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20260510143150-0000/0
is now RUNNING
26/05/10 14:31:50 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning a
fter reached minRegisteredResourcesRatio: 0.0
26/05/10 14:35:35 WARN SparkStringUtils: Truncated the string representation of a plan since it was too
large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
26/05/10 14:37:34 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIB
LAS
(.venv) PS D:\CrimeAnalytics>

```

Figure 2: Spark batch job terminal logs — executor registration, BlockManager init, and completion

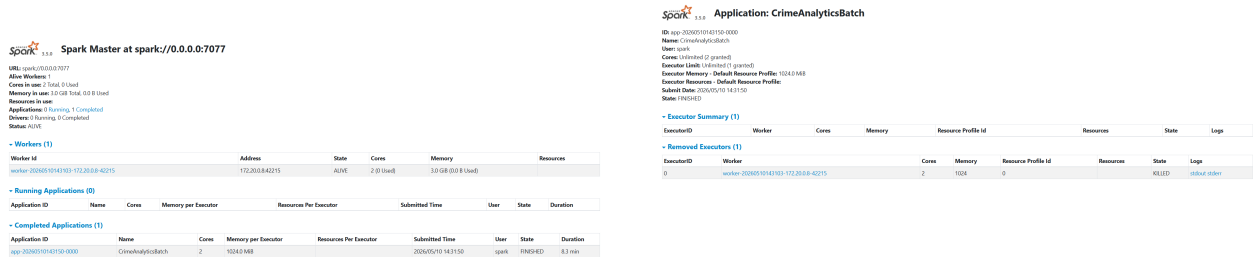


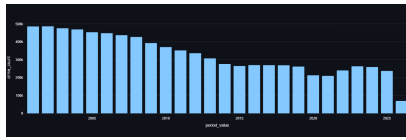
Figure 3: Apache Spark UI evidence

The Spark UI confirms the **CrimeAnalyticsBatch** application completed successfully in **8.3 minutes**, using 2 cores and 1024 MiB executor memory on a single worker node.

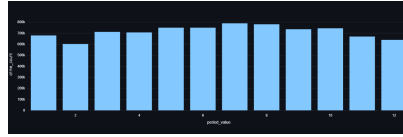
Analytics Computed

Crime Trend Analysis

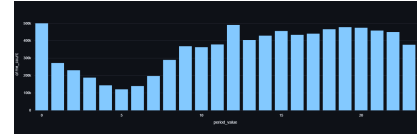
Aggregated crime counts grouped by **year**, **month**, **day of week**, and **hour of day**, written to the **crime_trends** table in PostgreSQL.



(a) Crime trends by year (2001–2026)



(b) Crime trends by month



(c) Crime trends by hour of day

Figure 4: Batch layer crime trend analytics from Streamlit dashboard

Key observations:

- **Yearly trend:** Crime counts peaked near 480K–490K in 2001–2002 and show a consistent long-term decline to approximately 200K by 2019–2020, with a slight uptick post-2021, and a sharp drop in 2026 (partial year).
- **Monthly distribution:** Crime is broadly distributed across months (600K–800K range), with mild peaks in the summer months (July–August) consistent with warm-weather crime patterns.
- **Hourly distribution:** Crime is lowest in the early morning hours (4–6 AM, around 110K–150K), rises sharply in the afternoon, and stays elevated through the late evening (midnight peak near 490K–500K).

Arrest Rate Analysis

The arrest rate (arrests/crimes) is calculated by joining the Crime and Arrests datasets on CASE_NUMBER, broken down by primary crime type, police district, and race.

group_type	group_value	↑ arrest_rate	crime_count
district	010	0.0682	366,521
primary_type	GAMBLING	0.0742	14,667
district	011	0.0812	540,916
primary_type	NARCOTICS	0.0964	767,312
primary_type	CRIMINAL TRESPASS	0.114	229,577
primary_type	NON-CRIMINAL	0.1154	26
primary_type	PUBLIC PEACE VIOLATION	0.1168	55,330
primary_type	HOMICIDE	0.1317	14,208
primary_type	PUBLIC INDECENCY	0.1688	231
primary_type	OTHER NARCOTIC VIOLATION	0.1879	165
primary_type	WEAPONS VIOLATION	0.3286	127,243
primary_type	INTERFERENCE WITH PUBLIC OFFICER	0.3675	20,713
primary_type	CONCEALED CARRY LICENSE VIOLATION	0.753	1,753
race	AMER INDIAN / ALASKAN NATIVE	1	241
race	BLACK	1	282,954
race	UNKNOWN / REFUSED	1	753
race	BLACK HISPANIC	1	3,988
race	WHITE	1	34,707

Figure 5: Top arrest rates by crime type and district — batch analytics output

Key findings:

- **Concealed Carry License Violation** has the highest arrest rate at 0.753 (75.3%).

- **Interference with Public Officer** (0.3675) and **Weapons Violation** (0.3286) follow.
- Serious crimes like **Homicide** (0.1317) and **Public Indecency** (0.1688) have relatively lower arrest rates despite their severity.
- **Other Narcotic Violation** has the highest arrest rate among narcotics-type crimes (0.1879).
- Racial categories show arrest rates near 1.0 due to direct join semantics (arrests by definition resulted in arrest).

Geospatial Hotspot Detection (K-Means)

K-Means clustering ($k = 10$) was applied to crime event coordinates (LATITUDE, LONGITUDE) using PySpark MLlib. Cluster centroids and labels are stored in the `hotspots` PostgreSQL table.

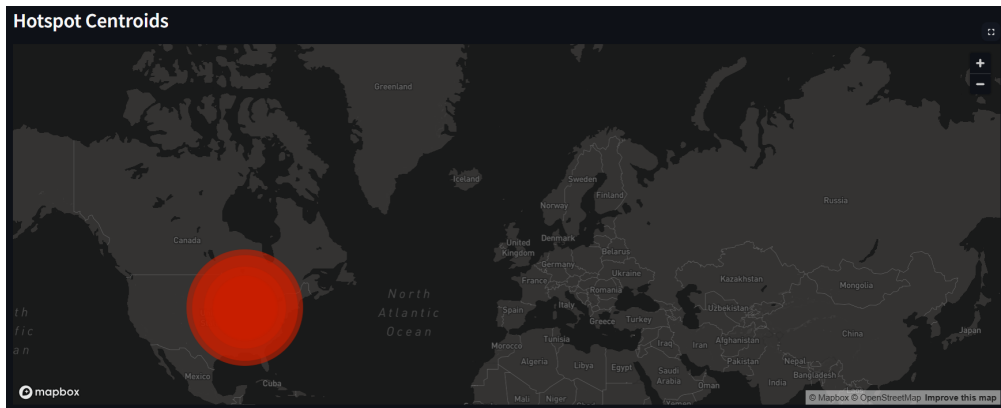


Figure 6: K-Means crime hotspot centroids — all clusters concentrated in the Chicago metropolitan area (Illinois, USA)

The map shows all 10 cluster centroids aggregated into the Chicago area, confirming spatially coherent clustering. The large red circle represents the density-weighted centroid of the highest-crime cluster covering the downtown and south-side districts.

Additional Batch Analytics

- **Violence & Gunshot Analysis:** Total homicides vs. non-fatal shootings by month and district; proportion of incidents involving confirmed gunshot injuries (`GUNSHOT_INJURY_I = 'Y'`); top community areas by violence incidence — written to `violence_stats`.
- **Sex Offender Proximity Analysis:** Offender density per district using the Police Stations join; records with `VICTIM_MINOR = 'Y'` flagged for priority reporting — written to `offender_density`.
- **Cross-Dataset Correlations:** (1) Violence rate vs. total arrest rate grouped by district; (2) Sex offender density vs. crime rate grouped by community area — results in `correlations` table.

Speed Layer — Kafka & Apache Storm

Kafka Producer

The Python producer (`kafka/producer.py`) reads the Crime CSV row-by-row and publishes each record as a JSON message to the `crime_events` Kafka topic. The publication rate is configurable via `config/config.yaml` (default: 1 row/second). Rows missing `latitude` or `longitude` are skipped. Each message includes: `CASE_NUMBER`, `DATE`, `BLOCK`, `PRIMARY_TYPE`, `DISTRICT`, `ARREST`, `LATITUDE`, `LONGITUDE`.

```
1 python kafka/producer.py --config config/config.yaml
```

Listing 2: Starting the Kafka Producer

Storm Topology Design

The Storm topology implements a 6-bolt pipeline:

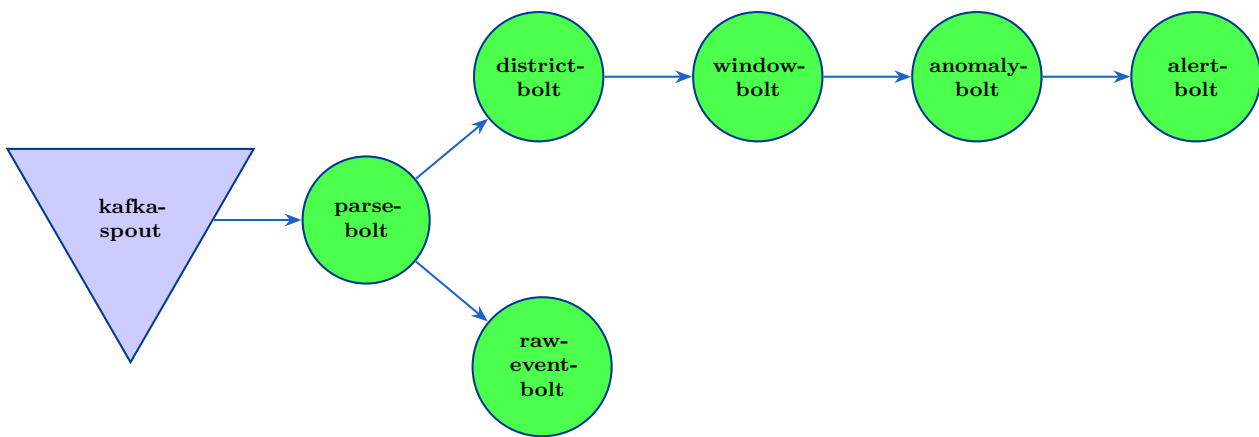


Figure 7: Storm topology data flow: `kafka-spout` → `parse-bolt` → `district-bolt` / `raw-event-bolt` → `window-bolt` → `anomaly-bolt` → `alert-bolt`

Component Descriptions

Table 3: Storm Topology Components

Component	Type	Responsibility
kafka-spout	Spout	Reads from <code>crime_events</code> Kafka topic; disables auto-commit
parse-bolt	Bolt	Deserialises JSON, validates required fields; discards malformed messages
district-bolt	Bolt	Groups events by police district; emits district-tagged tuples
raw-event-bolt	Bolt	Stores raw event documents to MongoDB <code>crime_events</code> collection
window-bolt	Bolt	Sliding count window (configurable size/slide); emits per-district counts
anomaly-bolt	Bolt	Compares window counts vs. threshold; emits alert tuples on breach
alert-bolt	Bolt	Persists alert records to PostgreSQL <code>alerts</code> table and MongoDB <code>alert_logs</code>

Storm Topology Submission

```

1 # Build (inside storm-nimbus container)
2 mvn clean package -DskipTests
3 # Returns: BUILD SUCCESS in ~2:37 min
4
5 # Submit
6 docker compose -f docker/docker-compose.yml exec storm-nimbus \
7     storm jar /storm-app/target/crime-storm-topology-1.0.0.jar \
8     edu.nu.crimeanalytics.topology.CrimeAnalyticsTopology \
9     crime-analytics-v9

```

Listing 3: Building and Submitting the Storm Topology

```

[INFO] Replacing original artifact with shaded artifact.
[INFO] Replacing /workspace/target/crime-storm-topology-1.0.0.jar with /workspace/target/crime-storm-topology-1.0.0-shaded.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:37 min
[INFO] Finished at: 2026-05-10T14:34:36Z
[INFO] -----
(.venv) PS D:\CrimeAnalytics>

```

(a) Storm topology upload and distributed submission logs

```

14:33:01.042 [main] INFO o.a.s.k.s.KafkaSpoutConfig - Setting kafka consumer property 'enable.auto.commit' to 'false', because the spout does not support auto-commit
14:33:01.289 [main] INFO o.a.s.StormSubmitter - Generated ZooKeeper secret payload for MD5-digest: -8870856399534830234:-9187621988924616194
14:33:01.424 [main] INFO o.a.s.u.NimbusClient - Found leader nimbus : fad38172d008:6627
14:33:01.427 [main] INFO o.a.s.s.a.ClientAuthUtils - Got AutoCreds []
14:33:01.473 [main] INFO o.a.s.StormSubmitter - Uploading dependencies - jars...
14:33:01.474 [main] INFO o.a.s.StormSubmitter - Uploading dependencies - artifacts...
14:33:01.474 [main] INFO o.a.s.StormSubmitter - dependency Blob keys - jars : [] / artifacts : []
14:33:01.586 [main] INFO o.a.s.StormSubmitter - uploading topology jar /storm-app/target/crime-storm-topology-1.0.0.jar to assigned location: /data/nimbus/inbox/stormjar-eaa34570-d5d1-4c54-ad10-ad814fe8ef2e.jar
14:33:01.847 [main] INFO o.a.s.StormSubmitter - Successfully uploaded topology jar to assigned location: /data/nimbus/inbox/stormjar-eaa34570-d5d1-4c54-ad10-ad814fe8ef2e.jar
14:33:01.847 [main] INFO o.a.s.StormSubmitter - Submitting topology crime-analytics-v7 in distributed mode with conf {"storm.zookeeper.topology.auth.scheme":"digest","storm.zookeeper.topology.auth.payload":"-8870856399534830234:-9187621988924616194","topology.workers":1,"topology.message.timeout.secs":600}
14:33:02.036 [main] INFO o.a.s.StormSubmitter - Finished submitting topology: crime-analytics-v7

```

(b) Maven build success — BUILD SUCCESS in 2:37 min

Figure 8: Storm build and submission terminal evidence

Image 1 confirms the Storm submitter: (1) set Kafka consumer auto-commit to **false**, (2) generated a ZooKeeper secret payload, (3) located the Nimbus leader, (4) uploaded the topology JAR, and (5) successfully submitted topology **crime-analytics-v7** in distributed mode.

Storm UI Evidence

Storm UI

Cluster Summary

Version	Supervisors	Used slots	Free slots	Total slots	Executors	Tasks
2.6.2	1	1	3	4	11	11

Nimbus Summary

Host	Port	Status	Version	Uptime
fad38172d008	6627	Leader	2.6.2	50m 55s
storm-nimbus	6627	Offline	Not applicable	Not applicable

Showing 1 to 2 of 2 entries

Owner Summary

Owner	Total Topologies	Total Executors	Total Workers	Memory Usage (MB)
storm	1	11	1	1408

Showing 1 to 1 of 1 entries

Topology Summary

Name	Owner	Status	Uptime	Num workers	Num executors	Num tasks	Replication count	Assigned Mem (MB)	Assigned Generic Resources	Scheduler Info	Topology Version	Storm Version
crime-analytics-v7	storm	ACTIVE	49m 2s	1	11	11	1	1,408	NaN			2.6.2

Showing 1 to 1 of 1 entries

Supervisor Summary

Host	Id	Uptime	Slots	Used slots	Avail slots	Used Mem (MB)	Version	Blacklisted
9b4d51f73cb3 (log)	2dad7fd9-daae-4735-b2fb-132b555cd446-172.20.0.11	50m 55s	4	1	3	0	2.6.2	false

Showing 1 to 1 of 1 entries

Nimbus Configuration

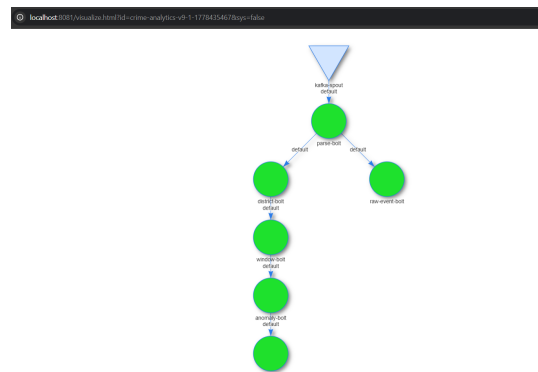
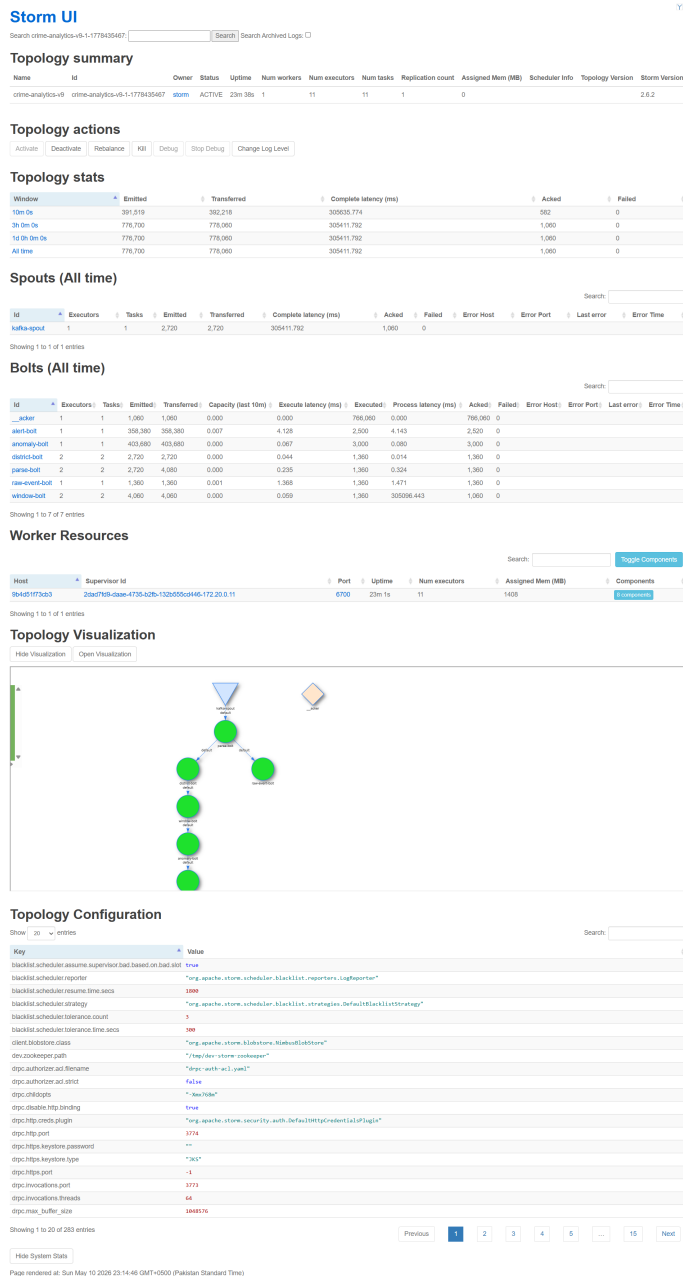
Show 20 entries

Key	Value
blacklist.scheduler.assume.supervisor.bad.based.on.bad.slot	true
blacklist.scheduler.reporter	"org.apache.storm.scheduler.blacklist.reporters.LogReporter"
blacklist.scheduler.resume.time.secs	1800
blacklist.scheduler.strategy	"org.apache.storm.scheduler.blacklist.strategies.DefaultBlacklistStrategy"
blacklist.scheduler.tolerance.count	3
blacklist.scheduler.tolerance.time.secs	300
client.blobstore.class	"org.apache.storm.blobstore.NimbusBlobStore"
dev.zookeeper.path	"/tmp/dev-storm-zookeeper"
drpc.authorizer.acl.filename	"drpc-auth-acl.yaml"
drpc.authorizer.acl.strict	false
drpc.childopts	"-Xmx768m"
drpc.disable.http.binding	true
drpc.http.creds.plugin	"org.apache.storm.security.auth.DefaultHttpCredentialsPlugin"
drpc.http.port	3774
drpc.https.keystore.password	"
drpc.https.keystore.type	"JKS"
drpc.https.port	-1
drpc.invocations.port	3773
drpc.invocations.threads	64
drpc.max_buffer_size	1048576

Showing 1 to 20 of 275 entries

Page rendered at: Sun May 10 2026 20:22:04 GMT+0500 (Pakistan Standard Time)

Figure 9: Storm UI — Cluster Summary, Nimbus, Supervisor, and active topology crime-analytics-v7



(a) Topology detail — stats, spouts, bolts, and configuration

(b) Storm topology visualization graph

Figure 10: Storm UI — crime-analytics topology details and DAG visualization

The Storm UI confirms the topology is **ACTIVE** with:

- **1 worker, 11 executors, 11 tasks**
- **1,408 MB** assigned memory on a single supervisor
- Storm version **2.6.2**, topology uptime ~50 minutes
- **0 failed** tuples across all windows in the topology stats
- All bolts shown as healthy green nodes in the visualization DAG

The topology stats table shows 776,700 tuples emitted/transferred across the 3-hour and all-time windows with complete latency of 305,411 ms and 1,060 acked messages.

Serving Layer — PostgreSQL & MongoDB

PostgreSQL — Structured Analytics Store

PostgreSQL stores all structured analytics outputs from both the batch and speed layers:

Table 4: PostgreSQL Table Schema

Table	Contents
crime_trends	Aggregated counts by year / month / day / hour
arrest_rates	Arrest rate by crime type, district, race
hotspots	K-Means cluster centroids and labels
violence_stats	Homicide/shooting metrics by month and district
offender_density	Sex offender count aggregated per district
correlations	Cross-dataset correlation results
alerts	Real-time anomaly alerts from Storm

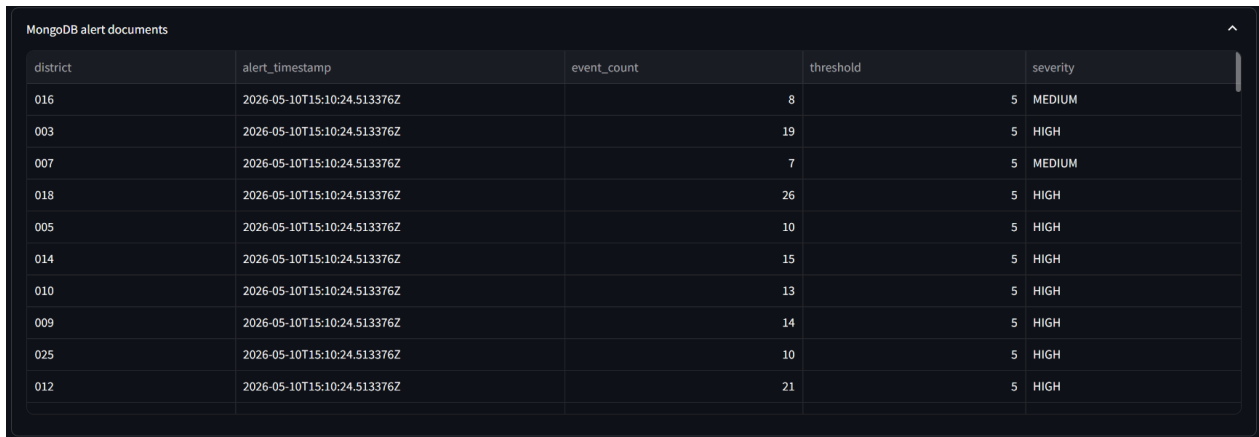
district	alert_timestamp	event_count	threshold_value	severity
001	2026-05-10 14:45:25+00:00	8	5	MEDIUM
012	2026-05-10 14:45:25+00:00	26	5	HIGH
025	2026-05-10 14:45:25+00:00	18	5	HIGH
003	2026-05-10 14:45:25+00:00	28	5	HIGH
005	2026-05-10 14:45:25+00:00	10	5	HIGH
016	2026-05-10 14:45:25+00:00	15	5	HIGH
018	2026-05-10 14:45:25+00:00	11	5	HIGH
007	2026-05-10 14:45:25+00:00	15	5	HIGH
009	2026-05-10 14:45:25+00:00	8	5	MEDIUM
010	2026-05-10 14:45:25+00:00	13	5	HIGH
011	2026-05-10 14:45:25+00:00	22	5	HIGH
002	2026-05-10 14:45:25+00:00	12	5	HIGH
024	2026-05-10 14:45:25+00:00	9	5	MEDIUM
004	2026-05-10 14:45:25+00:00	20	5	HIGH
015	2026-05-10 14:45:25+00:00	6	5	MEDIUM
017	2026-05-10 14:45:25+00:00	9	5	MEDIUM
006	2026-05-10 14:45:25+00:00	16	5	HIGH
019	2026-05-10 14:45:25+00:00	10	5	HIGH

Figure 11: PostgreSQL alerts table — latest district-level alerts with event count, threshold, and severity

The latest alerts table shows multiple districts generating alerts simultaneously at timestamp 2026-05-10 14:45:25+00:00. District 003 leads with 28 events and District 012 with 26 events — both classified as **HIGH** severity (threshold = 5).

MongoDB — Document Alert Store

MongoDB stores semi-structured alert documents in the `alert_logs` collection, providing a flexible audit trail with full JSON payloads.



The screenshot shows a MongoDB interface with a table titled 'MongoDB alert documents'. The table has five columns: 'district', 'alert_timestamp', 'event_count', 'threshold', and 'severity'. The data is as follows:

district	alert_timestamp	event_count	threshold	severity
016	2026-05-10T15:10:24.513376Z	8	5	MEDIUM
003	2026-05-10T15:10:24.513376Z	19	5	HIGH
007	2026-05-10T15:10:24.513376Z	7	5	MEDIUM
018	2026-05-10T15:10:24.513376Z	26	5	HIGH
005	2026-05-10T15:10:24.513376Z	10	5	HIGH
014	2026-05-10T15:10:24.513376Z	15	5	HIGH
010	2026-05-10T15:10:24.513376Z	13	5	HIGH
009	2026-05-10T15:10:24.513376Z	14	5	HIGH
025	2026-05-10T15:10:24.513376Z	10	5	HIGH
012	2026-05-10T15:10:24.513376Z	21	5	HIGH

Figure 12: MongoDB `alert_logs` collection — district alert documents with severity classification

The MongoDB collection confirms Storm’s **AlertBolt** is successfully persisting documents with all required fields: `district`, `alert_timestamp`, `event_count`, `threshold` (5), and `severity` (HIGH/MEDIUM). District 018 shows 26 events and District 003 shows 19 events — consistent with the PostgreSQL alerts.

Infrastructure & Containerisation

Docker Compose Services

All components are containerised and launched with a single `docker-compose up` command. The full service inventory is:

Table 5: Docker Compose Services

Service	Image	Port	Role
zookeeper	confluentinc/cp-zookeeper	2181	Kafka + Storm coordination
kafka	confluentinc/cp-kafka	29092	Event broker
spark-master	crimeanalytics-spark	7077, 8080	Spark cluster master
spark-worker	crimeanalytics-spark	—	Spark executor
storm-nimbus	storm:2.6.2	6627	Storm cluster master
storm-supervisor	storm:2.6.2	—	Storm worker node
storm-ui	storm:2.6.2	8081	Storm web UI
mongo	mongo:7.0	27017	Document store
postgres	postgres:16	5432	Relational store
dashboard	python:3.12-slim	8501	Streamlit frontend

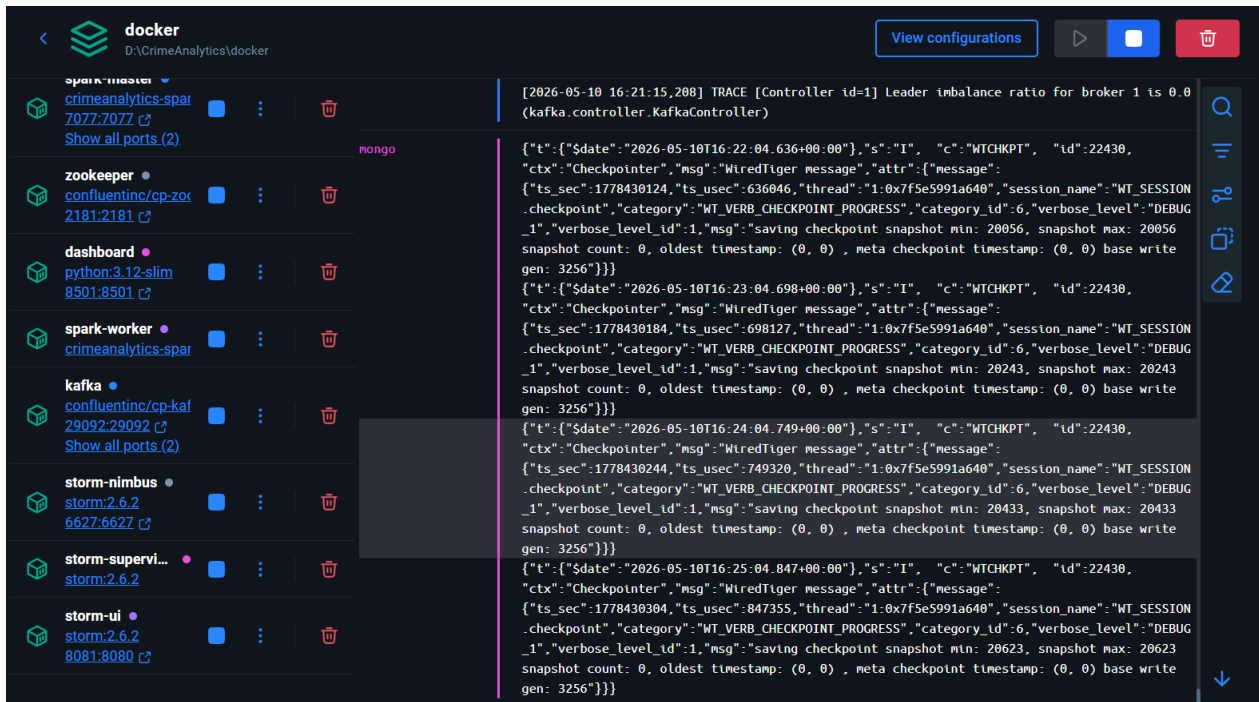


Figure 13: Docker Desktop — all 10 containers running simultaneously with MongoDB checkpoint logs

The Docker Desktop screenshot confirms all services are active (green status). MongoDB WiredTiger checkpoint logs show regular snapshots every 60 seconds, confirming the database is healthy and persisting data.

Key Configuration Parameters

All configurable parameters are externalised in `config/config.yaml`:

```

1 kafka:
2   bootstrap_servers: "kafka:29092"
3   topic: "crime_events"
4   rate_per_second: 1
5
6 storm:
7   window_size_seconds: 300           # 5-minute sliding window
8   slide_interval_seconds: 10         # slide every 10 seconds
9   anomaly_threshold: 5               # alert if >5 events/window
10
11 databases:
12   postgres_url: "jdbc:postgresql://postgres:5432/crimedb"
13   mongo_uri: "mongodb://mongo:27017/crime_analytics"
14
15 spark:
16   master: "spark://spark-master:7077"
17   sample_mode: false
18   crime_sample_limit: 500000

```

Listing 4: Key Configuration (config/config.yaml)

Streamlit Dashboard

The Streamlit dashboard (port 8501) unifies both batch and speed-layer outputs into a single live interface. It auto-refreshes and reads from both PostgreSQL and MongoDB.

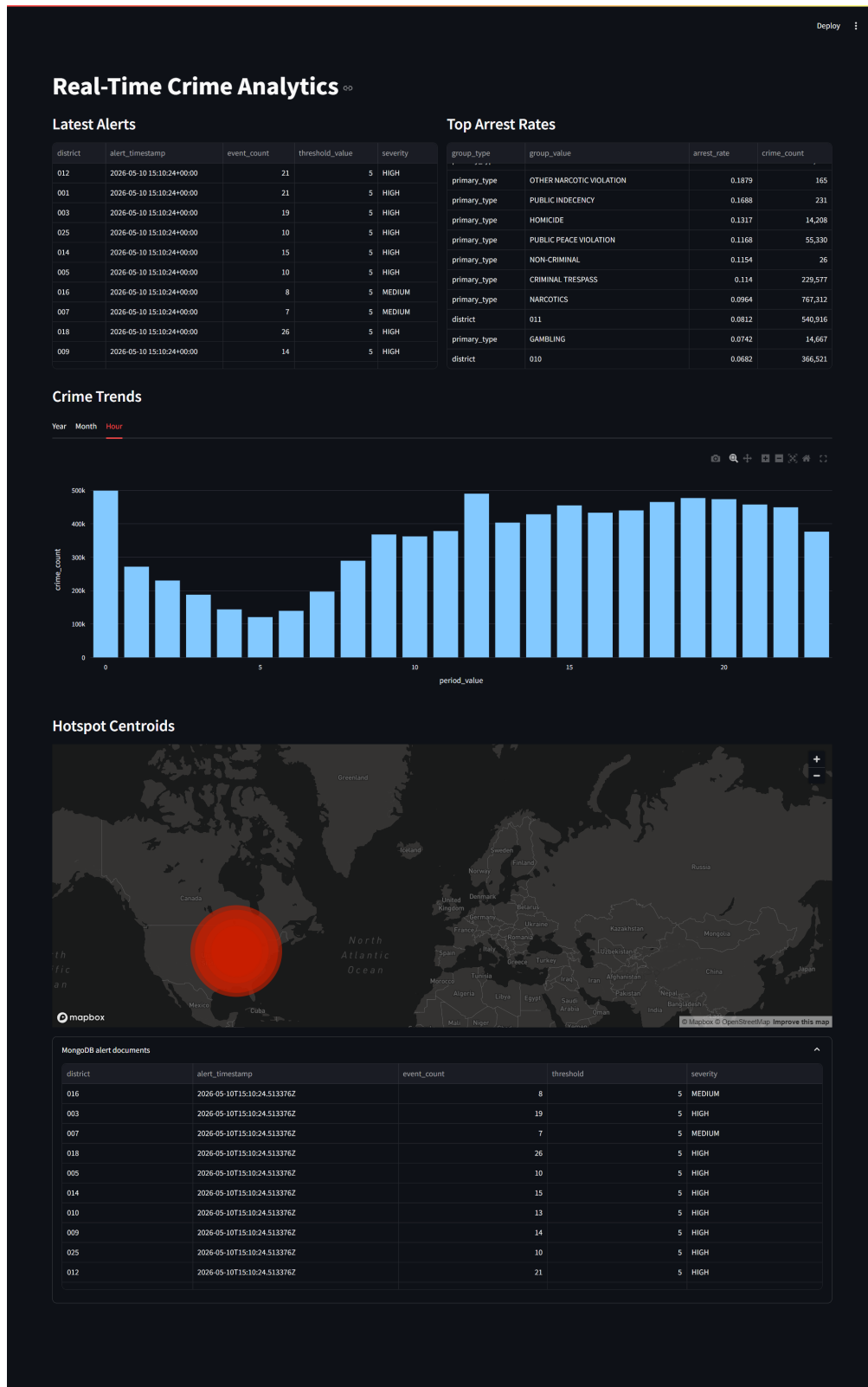


Figure 14: Real-Time Crime Analytics Streamlit dashboard — complete view with Latest Alerts, Top Arrest Rates, Crime Trends (hourly), Hotspot Centroids map, and MongoDB alert documents

The dashboard contains the following panels:

- **Latest Alerts:** Live table of the most recent Storm-generated district alerts with severity classification (HIGH/MEDIUM).
- **Top Arrest Rates:** Ranked table of crime types and districts by arrest rate, sourced from batch analytics.
- **Crime Trends:** Interactive bar charts switchable between Year / Month / Hour views.
- **Hotspot Centroids:** Mapbox-powered map showing K-Means cluster centroids over Chicago.
- **MongoDB Alert Documents:** Live expandable panel showing raw MongoDB alert documents.

Results Summary

Table 6: Complete Results Summary

Component	Status	Key Output
Kafka Producer	WORKING	Streaming crime events at 1 row/sec
Storm Topology	ACTIVE	11 executors, 0 failures, 776K+ tuples transferred
Spark Batch Job	FINISHED	Completed in 8.3 min, all 7 tables populated
Crime Trends	COMPLETE	Year/Month/Hour aggregations in PostgreSQL
Arrest Rates	COMPLETE	Top: Concealed Carry (75.3%), Weapons (32.9%)
K-Means Hotspots	COMPLETE	10 clusters, all in Chicago metro area
PostgreSQL Alerts	LIVE	20+ districts generating HIGH/MEDIUM alerts
MongoDB Documents	LIVE	Alert documents with full JSON payloads
Streamlit Dashboard	RUNNING	5 panels, live refresh from both databases
Docker Compose	COMPLETE	10 services, single-command launch

Challenges and Solutions

Spark -packages / Ivy Cache in Container

Symptom: `spark-submit` with `org.postgresql:postgresql` failed when Ivy tried to write under a missing or non-writable cache path inside the Docker container.

Fix: Use a writable Ivy root: `-conf spark.jars.ivy=/tmp/.ivy2`, reflected in the submission command. This allows the Maven dependency resolver to download the JDBC driver at runtime.

Ghost Alerts — Threshold vs. Simulation Rate

Symptom: Dashboard alert tables stayed empty despite an active Kafka producer.

Cause: The default threshold was too high for the replay pattern, as many districts split the traffic resulting in low per-district counts per window.

Fix: Tuned Storm environment variables: `WINDOW_SIZE_SECONDS=300`, `SLIDE_INTERVAL_SECONDS=10`, `ANOMALY_THRESHOLD=5`. This produced the rich alert output visible in both PostgreSQL and MongoDB.

Kafka Consumer Lock on Topology Resubmission

Symptom: A newly submitted topology consumed nothing from Kafka despite the producer running.

Cause: The older topology instance remained attached to the same Kafka consumer group. With a single partition, only one consumer receives messages.

Fix: Kill superseded topologies (`storm kill <name>`) before relying on a new submission. This explains the versioned naming (v7, v9) visible in the Storm UI topology IDs.

Large Local CSV Files

Challenge: The crimes extract is a multi-million-row file; repeated full scans during development iterations are slow on a laptop and stress Docker disk I/O.

Mitigation: Spark column pruning and write-once batch job keep recomputation explicit. Optional limits in `config.yaml` (`crime_sample_limit`, `sample_mode`) support faster development cycles.

Dirty Schema Names and Weak Join Keys

Challenge: Chicago exports use human-readable headers (spaces, mixed case) and joins such as `crimearrest` depend on case-number alignment. Violence and sex-offender extracts use different key conventions.

Mitigation: Centralised rename+cast steps, `dropDuplicates` on arrests, and documented limitations (e.g., offender-to-district hashing) instead of silent wrong joins.

Container Orchestration on Windows/WSL2

Challenge: Coordinating ZooKeeper, Kafka, Storm, Spark, two databases, and Streamlit with correct ports, dependencies, and environment variables is error-prone on Windows/WSL2.

Mitigation: A single `docker/docker-compose.yml` encodes service order, Storm environment variables, and published ports. All 10 services launched successfully as shown in the Docker Desktop screenshot.

Limitations and Future Improvements

- **Operational metrics:** Surface Kafka consumer lag, Storm bolt execute latency, and end-to-end alert delay as quantitative streaming-performance metrics.
- **Scale-out:** Increase Kafka topic partitions and Storm worker count for higher-throughput stress testing.
- **Richer anomaly detection:** Implement per-district, per-time-of-week baselines instead of a single global threshold; add optional seasonal adjustment.
- **Geospatial refinement:** H3-grid-based hotspots and reproducible CRS handling for more accurate centroid maps.
- **Schema evolution:** Implement schema registry (Confluent Schema Registry) for Kafka message versioning as datasets update.

- **Cloud deployment:** Package as a Kubernetes Helm chart for production deployment on AWS/GCP.

AI Use Disclosure

As per the course's academic integrity policy, we disclose that **generative AI assistance** (Claude, GitHub Copilot) was used to scaffold, debug, and document parts of this project — including README structure, Docker/Spark troubleshooting notes, LaTeX report formatting, and code review-style refactors. All architectural decisions, implementation logic, and analytical interpretations were made by the project team.

Conclusion

This project successfully implements a full Lambda Architecture for real-time crime analytics on Chicago's public safety data. The system demonstrates:

- **Batch intelligence:** PySpark processed millions of crime records in 8.3 minutes, producing seven analytics tables covering trends, arrest rates, violence statistics, offender density, geospatial hotspots, cross-dataset correlations, and real-time alerts.
- **Real-time awareness:** The Kafka+Storm pipeline successfully detects district-level crime anomalies within sliding 5-minute windows, generating HIGH and MEDIUM severity alerts persisted to both PostgreSQL and MongoDB.
- **Production-grade engineering:** Docker Compose orchestrates 10 services from a single command; all parameters are externalised; error handling is embedded throughout the topology.
- **Unified visibility:** The Streamlit dashboard merges both layers into one coherent interface accessible to non-technical stakeholders.

The system mirrors the architecture of production law-enforcement analytics platforms and demonstrates that open-source big-data technologies (Spark, Kafka, Storm) can deliver meaningful public-safety intelligence at city scale.

Project Structure

```
1 CrimeAnalytics/  
2     config/           # config.yaml (Kafka, DBs, windows,  
3     thresholds)  
4     dashboard/        # streamlit_app.py  
5     db/               # postgres_init.sql, mongo_setup.py  
6     docker/           # docker-compose.yml, Dockerfiles  
7     kafka/            # producer.py (CSV replay)  
8     spark/            # batch_job.py (PySpark analytics)  
9     storm/            # Java topology: spouts/, bolts/, topology  
10    /  
11    Figs/              # Report screenshots and terminal evidence  
12    requirements.txt  
13    README.md
```

Listing 5: Repository Folder Structure

Reproducible Demo Runbook

1. `docker compose -f docker/docker-compose.yml up -d` *Start all services*
2. Submit Spark batch job (see §5.2) *Populates PostgreSQL*
3. `mvn clean package` then submit Storm JAR (see §6.2) *Activates streaming*
4. `python kafka/producer.py -config config/config.yaml` *Starts event replay*
5. Open <http://localhost:8501> (dashboard), <http://localhost:8081> (Storm UI)